

Where the Bytes Go: Measuring the von Neumann Bottleneck in On-Device LLM Inference, and What Processing-in-Memory Would Change

Anish Talla

Independent student researcher
anish.talla99@gmail.com

Abstract—Generating one token from a language model at batch size 1 means reading every weight once, which makes on-device inference a direct exercise of the von Neumann bottleneck. We measure it on an Apple M4 laptop with 16 GB of unified memory: four Qwen2.5 models in up to seven weight formats (23 configurations), on GPU and CPU, across thread counts, context lengths and batch sizes. Sixteen-bit decode reaches 83 to 95% of the measured memory bandwidth divided by the bytes read per token, a line with no fitted parameters. Across weight formats decode speed changes by up to $3.0\times$ while prefill changes by at most 17%. The small-batch decode kernels have a compute ceiling about $6\times$ below the one prefill reaches, which puts the decode ridge near 4.4 FLOP per byte and 4-bit decode at about three quarters of it; batching 2 to 8 sequences therefore buys little. We fit a latency model on two model sizes and test it on the others. On a blind 7B test, bandwidth over bytes is off by 9.5%, a model with a bandwidth efficiency and a per-layer overhead by 8.6%, and one with a per-format constant by 3.3%; the remaining error is sorted by weight format. We then project onto published parameters of commercial processing-in-memory parts. They compute only in 16-bit floating point and are provisioned with just enough arithmetic for that. A 4-bit host is already 2.0 to $2.9\times$ faster than a 16-bit one on our machine, so a 16-bit PIM with a fourfold internal bandwidth advantage nets 1.2 to $1.4\times$ over it, at about three times the memory. A 4-bit PIM with the same arithmetic throughput would be no faster. For on-device inference, today’s PIM parts are missing low-bit arithmetic, and about three times as much of it per byte of bandwidth.

I. INTRODUCTION

In 1978 Backus described a computer as a processor, a store, and a tube between them that carries one word at a time, and proposed “to call this tube the von Neumann bottleneck” [1]. The stored-program organization he was describing [2] has not changed. What has changed is the ratio between how fast processors compute and how fast memory can feed them. Wulf and McKee named the consequence the memory wall [3]; their argument was about access latency, and ours is about bandwidth. Gholami et al. put current numbers on the bandwidth side: peak server FLOPS have grown $3.0\times$ every two years, DRAM bandwidth $1.6\times$ [4].

Autoregressive language-model inference is a workload that sits directly on this bottleneck. To produce one token at batch size 1, every weight matrix is read from memory once and used for one matrix-vector product. The arithmetic per byte is tiny, so the time per token is set by how fast the weights cross the memory interface [5], [4], [6]. Processing-in-memory

(PIM) computes inside or next to the DRAM [7], where the bandwidth available to the compute units is several times higher than at the pins: Samsung reports four times for its HBM-PIM [8]. Samsung, SK hynix and UPMEM have all built such parts [9], [10], [11], and recent architecture papers evaluate them for LLM serving in the datacenter [12], [13], [14], [15].

This paper looks at the other end of the scale: one person, one laptop, one sequence, 4-bit weights. That is the case a local inference tool such as llama.cpp serves [16], and it is a regime the PIM-for-LLM papers we read in full do not cover. IANUS, NeuPIMs and CENT use datacenter GPUs as the baseline, 16-bit weights, and simulators as the instrument [14], [13], [15]. We use a stopwatch first. We measure a family of four models at up to seven quantization levels on an Apple M4 laptop, fit a three-constant analytical model to 14 of the 23 configurations, and test it on the other 9. Only then do we use the model to ask what today’s commercial PIM parts would change.

We do not claim that decode being memory-bound is a new finding; Section II lists who showed it first. The contributions are narrower:

- **A measured characterization** of LLM inference on a unified-memory consumer SoC across 23 model and quantization configurations, covering GPU and CPU backends, thread count, context length and batch size, with all raw data and scripts released.
- **The error of the first-order model.** The two-term latency model most people reason with (memory time plus compute time [5], or their maximum [17]) is checked against held-out measurements. We report where it holds and where it fails. The failures are sorted by weight format, and the small-batch decode kernels turn out to have a compute ceiling several times below the one prefill reaches, which puts 4-bit decode close to the ridge of the roofline.
- **The datatype mismatch.** Commercial PIM parts compute in FP16 or BF16 only [18], [19]. Weight quantization already recovers a large factor on a bandwidth-bound baseline. We quantify how much of PIM’s internal-bandwidth advantage is left once the baseline is allowed to run 4-bit weights and the PIM part is not, and show that the published parts are compute-balanced for 16-bit weights, so a 4-bit PIM would need several times their

arithmetic throughput to be any faster.

II. BACKGROUND AND RELATED WORK

A. Why decode is bandwidth-bound

A decoder-only transformer with N parameters needs about $2N$ floating-point operations per generated token [5]. It also needs every one of those parameters to be present at the arithmetic units, so W bytes of weights cross the memory interface per token, where W is the size of the weights in their stored format. Pope et al. call the two costs compute time and memory time, and note that “at small batch sizes and sequence lengths, the time to load weights dominates” [5]. In the vocabulary of the roofline model [17], the operational intensity of batch-1 decode is $2N/W$ FLOP per byte: 1 for 16-bit weights, and about 3.2 for the Q4_K_M files we measure, which hold close to 5 bits per weight. The ridge point of a current GPU is two orders of magnitude higher: 153 FLOP per byte for an A100 by its datasheet figures [20]. Prompt processing is different. All prompt tokens share one read of the weights, the products become matrix-matrix, and the intensity rises with the number of tokens processed together [4]. Batching sequences does the same for decode [5], and datacenter serving systems are built around it [21], [22]. A single user has at most the small batch that speculative decoding creates.

Weight-only quantization is the standard response on the software side. GPTQ reports $3.25\times$ faster generation than FP16 on an A100 [23], and AWQ reports more than $3\times$ on desktop and mobile GPUs [24]. We read those speedups as a sign of a bandwidth-bound workload, since weight-only quantization reduces bytes and not arithmetic; that reading is ours, not the authors’.

B. The energy side

Horowitz’s 45 nm estimates put one 64-bit DRAM access at 1.3 to 2.6 nJ against 3.7 pJ for a 32-bit floating-point multiply [25]; the often-quoted 640 pJ for a 32-bit DRAM read comes from the table reproduced by Han et al. [26]. These are rough figures for an old process and an old DRAM generation. A more recent model by O’Connor et al. gives 3.92 pJ/bit for an HBM2 access, of which 2.24 pJ/bit is moving the data from the sense amplifiers to the package bumps and 0.3 pJ/bit is crossing the interposer [27]. About 65% of the access energy is movement. Near-bank PIM can avoid most of that share. The row activation remains.

C. Commercial PIM parts

Three PIM products have public characterizations. **Samsung HBM-PIM** places 16-lane FP16 multiply-accumulate units at the bank level of an HBM2 stack and reports $4\times$ the processing bandwidth of the off-chip interface, $11.2\times$ on memory-bound kernels and $3.5\times$ on applications [8], [9], [18]. **SK hynix GDDR6-AiM** [10] has 32 processing units per chip, computes in BF16, and is rated at 1 TFLOPS per chip with 64 GB/s at the pins [19]; the IANUS paper, co-authored by SK hynix engineers, gives its internal bandwidth as 1024 GB/s per chip [14]. **UPMEM** puts general-purpose 32-bit cores next to

DDR4 banks. Its aggregate internal bandwidth is high, but the cores have no full hardware multiplier and emulate floating point, so a 32-bit multiply runs at about 10 million operations per second per core [11]. All three trade DRAM density for logic: CENT quotes 25 to 50% of conventional capacity for UPMEM and HBM-PIM dies and 75% for AiM [15].

D. PIM for LLM inference

AttAcc moves the attention layer of batched serving into PIM and leaves the fully connected layers on the host accelerator [12]. NeuPIMs pairs an NPU for matrix-matrix work with PIM for matrix-vector work and includes a roofline analysis showing GPT-3 generation is memory-bound on an A100 [13]. IANUS couples an NPU with AiM in a unified memory system and reports $6.2\times$ over an A100 on GPT-2 at batch 1 [14]. CENT removes the GPU entirely and serves Llama 2 from CXL-attached AiM-style devices, with $2.3\times$ the throughput and $2.9\times$ lower energy than a GPU baseline [15]. PIM-AI, a vendor preprint, is the one proposal we found that targets LPDDR-class mobile memory [28].

E. What is left to do

The three PIM-for-LLM papers we read in full, IANUS, NeuPIMs and CENT, have three things in common. They evaluate by simulation, with real PIM silicon appearing only in a reduced-speed FPGA prototype used for functional checks [14], [19]. Their baseline is a datacenter GPU running 16-bit weights. And none of them evaluates the 4-bit weight formats common in on-device inference, which the commercial PIM parts cannot execute natively. We read only the abstracts of AttAcc and PIM-AI and make no claim about them. The work closest to ours in spirit is Gholami et al., who argue the memory-wall thesis from hardware trends and operation counts and show measured CPU latency for BERT and GPT-2 [4], and Yuan et al., who build a roofline-based analysis tool for LLM inference [6]. Gholami et al. do not fit a latency model and report its error against measurements, and they do not cover quantized models or a unified-memory SoC; we read only the abstract of Yuan et al. and cannot say the same of them. DAMOV sets the methodological precedent of characterizing data movement on real workloads before evaluating near-data processing [29]; we follow that order here.

III. METHODOLOGY

A. Platform

All measurements were taken on one machine: a MacBook Air (15-inch, M4, 2025) with 16 GB of unified memory, macOS 26.6.2. The M4 has four performance and six efficiency CPU cores and a 10-core GPU that share one memory, rated by Apple at 120 GB/s [30]. Apple does not publish a peak FLOPS figure for the GPU, so we do not use one. Every compute ceiling in this paper is measured.

Unified memory matters for the experiment design. The CPU and the GPU are two very different processors behind the same memory interface. If decode is limited by that interface, the two should produce tokens at similar rates even though

their compute throughput differs by a large factor. Section IV tests this.

B. Software and models

We use llama.cpp [16] at commit a894dae, built twice: once with the Metal backend for GPU runs, and once with Metal disabled so that CPU runs cannot offload anything. Timing comes from its llama-bench and llama-batched-bench tools.

The models are Qwen2.5-Instruct at 0.5B, 1.5B, 3B and 7B parameters [31]. One family keeps the architecture fixed while size varies. For the three smaller models we download the 16-bit GGUF file and produce Q8_0, Q6_K, Q5_K_M, Q4_K_M, Q3_K_M and Q2_K variants locally with llama-quantize. The 16-bit 7B model does not fit in 16GB, so for 7B we use the published Q8_0 and Q4_K_M files. That gives 23 configurations. Quantization quality is not evaluated here; only speed depends on it in this study, and only through the number of bytes and the cost of the kernels.

C. Bytes read per token

The quantity that matters is not file size. GGUF files mix tensor types, and the token-embedding table is accessed by row lookup, one row per token. We read every tensor's byte count from the file and define W as the sum over all tensors except the embedding table. The published Qwen2.5 GGUF files store the output projection as a separate tensor even where the original model ties it to the embedding, so the output projection is always counted. For the 0.5B model the difference is large: the 16-bit file is 1.26 GB but W is 0.99 GB.

D. Measurements

Each configuration is timed for prefill of a 512-token prompt and for decode of 128 tokens, five repetitions each. We report the mean and standard deviation of tokens per second. The machine is passively cooled, so every configuration is preceded by 60s of idle time, and a configuration whose standard deviation exceeds 5% of its mean is repeated after a longer pause, at most twice; Section IV-G explains why this matters. Around this core sweep we vary one factor at a time:

- **Backend:** GPU (all layers offloaded) and CPU on the four performance cores.
- **Threads:** 1, 2, 4, 6, 8 and 10 CPU threads, Q4_K_M, each count measured separately (256-token prompt, 64 generated tokens, three repetitions).
- **Context:** decode with 0, 1024, 4096 and 8192 tokens already in the KV cache, Q4_K_M and Q8_0.
- **Batch:** 1 to 64 parallel sequences, 128-token prompts and 64 generated tokens each, for Q4_K_M, Q8_0 and the largest format that fits.

Memory bandwidth is measured separately with STREAM-style copy and triad kernels over 640MB arrays at 1 to 10 threads. The highest figure, 107 GB/s (the best of nine timed repetitions of the copy kernel), is used as the bandwidth BW everywhere below; it is 89% of the rated 120 GB/s.

Every result file stores the exact command, the llama.cpp commit, the power source, the battery level and the low-power-mode flag at the time of the run; files from the final pass also store the busiest unrelated process and its CPU share, and that pass waits for a quiet machine before each configuration.

E. Analytical model

For one decode step serving B sequences that each have L tokens in the KV cache,

$$\text{bytes}(B, L) = W + B \kappa L, \quad (1)$$

$$\text{flops}(B, L) = B (2N_a + 4n_\ell d L), \quad (2)$$

$$t_{\text{step}} = \max\left(\frac{\text{bytes}}{\eta_m BW}, \frac{\text{flops}}{C}\right) + \tau n_\ell, \quad (3)$$

where $\kappa = 2n_\ell n_{kv} d_h \cdot 2$ bytes is the 16-bit KV cache per token, N_a is the parameter count without the embedding table, n_ℓ the number of layers, d the hidden size, n_{kv} and d_h the number and size of KV heads. This is the memory-time and compute-time decomposition of Pope et al. [5] on one device, in roofline form [17], with a fixed per-layer overhead added. We also fit an additive form in which the maximum is replaced by a sum.

Three constants are fitted. C is the effective compute throughput, taken as the median over prefill runs of tokens per second times FLOPs per prompt token; prompt tokens skip the output projection, because logits are computed only for the last position. η_m is the fraction of the measured bandwidth that decode kernels achieve, and τ is the per-layer overhead of launching kernels; both are fitted by least squares on relative decode error. All architectural quantities are read from the GGUF files, not fitted.

Held-out validation. The constants are fitted on the 0.5B and 3B models only. The choice between the roofline and additive forms is made on training error. The 1.5B and 7B models are never used for fitting, but they are not equally clean tests, and Section V says exactly what was seen when.

F. Projection to PIM

For a PIM device we keep the same equations and change where the weight term is paid. Matrix-vector products over the weights run on the in-memory units at the device's internal bandwidth BW_{int} and compute rate C_{pim} . Each sequence in a batch is a separate matrix-vector product, and a bank-level unit streams the weight rows through its multiply-accumulate lanes for each one, so the weight term scales with B :

$$t_{\text{pim}} = B \cdot \max\left(\frac{W_{\text{pim}}}{r \eta_m BW}, \frac{2N_a}{C_{\text{pim}}}\right) + t_{\text{rest}}, \quad (4)$$

where $r = BW_{\text{int}}/BW_{\text{ext}}$ is the device's internal-to-external bandwidth ratio applied to the host's own memory technology, C_{pim} follows from the device's published ratio of compute throughput to internal bandwidth, and t_{rest} is the part of the measured 16-bit host step that is not the weight read, never taken below the fitted per-layer overhead τn_ℓ . W_{pim} is the weight size in a format the device can execute. For HBM-PIM and AiM that is 16 bits per weight regardless of how the baseline stores the model. Every device parameter is taken

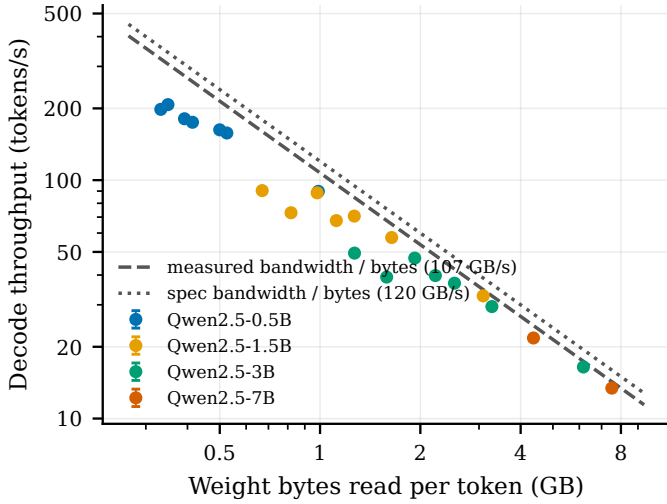


Fig. 1. Decode throughput against weight bytes read per token, batch 1, GPU. The dashed line is the measured memory bandwidth divided by the bytes, with no fitted parameters. Error bars are one standard deviation over five repetitions.

from a vendor publication or a peer-reviewed paper, listed with its source in Table II. Parameters that are not published are not guessed; where one is needed we sweep it over a stated range. The projection ignores command scheduling, row-activation timing, the capacity lost to PIM logic, and host transfers for the operators that stay outside the memory. It is a first-order estimate of what the memory side could deliver.

IV. MEASUREMENTS

A. Decode follows the bytes; prefill does not

Fig. 1 plots decode throughput against W for all 23 configurations. The dashed line is BW/W with the measured bandwidth: the rate at which the memory could deliver complete copies of the weights if nothing else took time. It has no free parameters. The 16-bit models reach 83%, 94% and 95% of it at 0.5B, 1.5B and 3B, and 7B in Q8_0 reaches 94%. These percentages inherit the uncertainty of the denominator. Our copy kernel gives 103 to 107 GB/s and the triad kernel 97 to 98 GB/s; we use the highest figure. An independent check agrees with it: the slope of 16-bit step time against W across the three sizes corresponds to 104 GB/s.

Fig. 2 separates the two phases. Within one model size, changing the weight format changes decode speed by a factor of 2.3 to 3.0. Over the same formats, prefill speed moves by 8 to 17%. FLOPs per token are the same in both phases. What differs is how many tokens share each read of the weights: one in decode, 512 in prefill.

B. Two compute ceilings

A roofline needs a compute ceiling, and this GPU has two, because llama.cpp uses different kernels for the two phases. For prefill we sweep the prompt length. Achieved FLOP/s rises from 2.2 TFLOP/s at 32 tokens to a peak of 2.9 TFLOP/s at 512 tokens, and at the longest prompt it is 98% of the peak.

For decode, the batch experiment of Section IV-F shows total throughput stalling at 0.38 to 0.49 TFLOP/s in every retained run (1.5B to 7B, three formats). The matrix-vector kernels used at small batch top out about 6× below the prefill peak.

That puts the ridge point of the decode path at about 4.4 FLOP per byte (Fig. 3). Sixteen-bit decode, at 1 FLOP per byte, is far to its left. Q4_K_M decode is not: our 1.5B to 7B files work out to 3.2 FLOP per byte, under the ridge but close to it. On this machine 4-bit decode is where the workload stops being purely bandwidth-bound.

C. Quantization helps less than its byte count

Going from 16-bit to Q4_K_M shrinks W by 3.15× for the 1.5B model and 3.21× for 3B, but decode speeds up by 2.71× and 2.86×. Q4_K_M reaches 81%, 84% and 89% of its own bandwidth line at 1.5B, 3B and 7B. The two smallest formats do worse: Q2_K and Q3_K_M reach only 56 to 58% of their lines at 1.5B and 3B. At 1.5B and 3B, Q3_K_M is both smaller and slower than Q4_K_M (73.0 against 88.5 tokens/s, and 39.2 against 47.1), for a saving of 17 and 18% of the bytes.

The 0.5B model does not follow this pattern, and the reason is a labelling one. Its hidden size is not a multiple of the 256-weight block that the K-quant formats use, so llama-quantize falls back to other tensor types for many matrices. Its “Q4_K_M” file holds 6.35 bits per active weight, against 4.99 at 3B. Format names are not comparable across the smallest size, which is why we always plot measured bytes.

D. Two processors, one memory

The CPU and GPU of the M4 share one memory interface. Across 23 configurations, CPU decode on the four performance cores runs at a median 76% of GPU decode (range 67 to 92%), and CPU prefill at a median 33% of GPU prefill (range 22 to 48%); see Fig. 4. The prefill figure needs a caveat. The default CPU build sends large matrix products to Apple’s Accelerate library, which does not run them on the same execution units as decode. With Accelerate disabled, CPU prefill on the 3B model is 8 to 33% of the GPU’s, depending on format, while CPU decode is unchanged at 71 to 75%. The same four cores deliver about three quarters of the GPU’s decode rate and between a twelfth and a third of its prefill rate, depending on format.

Four cores do not saturate the interface. Sixteen-bit decode on four performance cores reaches 63 to 67% of the bandwidth line, against 83 to 95% on the GPU, for identical bytes. Fig. 5 gives the thread curve for Q4_K_M. Relative to one thread, decode reaches 1.7×, 1.8×, 2.3×, 2.4× and 1.7× at 2, 4, 6, 8 and 10 threads, and prefill 1.8×, 3.3×, 3.7×, 4.0× and 1.4× (medians over 3 model sizes). In the median, decode keeps gaining when efficiency cores join, peaks at 8 threads (at six for 1.5B and 7B, by a small margin) and falls back at ten at every size. The prefill medians hide a split: the 0.5B model stops gaining at four threads, and the 7B model is still gaining at ten. With all ten cores in use, any other activity on the machine turns one thread into a straggler, and one ten-thread run collapsed to a tenth of the single-thread rate. At its best thread count, CPU

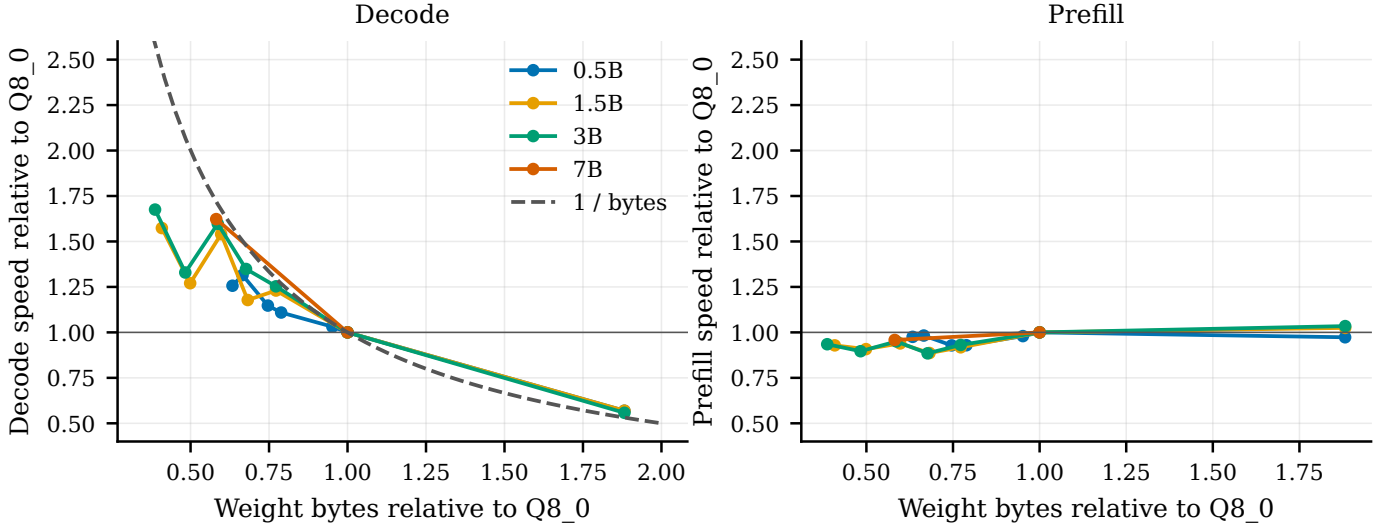


Fig. 2. Speed against weight size, both normalized to Q8_0 of the same model, on the same vertical scale. Decode follows $1/\text{bytes}$ between 16-bit and Q8_0 and falls below it for the smaller formats. Prefill barely moves.

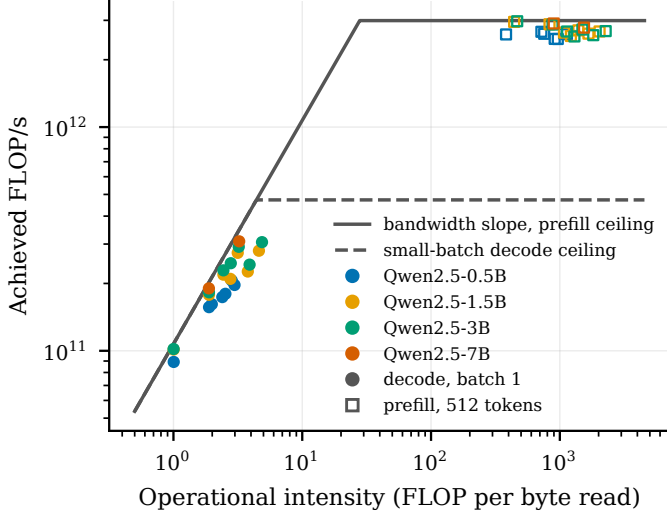


Fig. 3. Roofline of the M4 GPU from measured quantities. The slope is the measured bandwidth. The upper ceiling is the peak of the prompt-length sweep; the lower one is the plateau of the small-batch decode kernels. Prefill points sit 512 times to the right of decode points of the same model because 512 tokens share each read.

decode reaches 89 to 105% of GPU decode across the three sizes, and 95% for the 7B model. For a model large enough that fixed overheads are small, a CPU with a fraction of the GPU’s compute produces tokens at nearly the same rate, which is what a shared bandwidth limit predicts. Multi-threaded CPU runs are the noisiest data in the paper, with spreads up to 54% after three attempts (the collapsed ten-thread run).

E. The KV cache

With L tokens in the cache, each decode step also reads κL bytes of keys and values. If bytes were all that mattered, decode speed at depth L relative to an empty cache would be

$W/(W + \kappa L)$, a prediction with no fitted constants (Fig. 6). Its mean error over all depth measurements is 4.1%, against 10.7% for predicting no slowdown at all, so it explains most of the effect. It is not uniform: 2.3%, 1.8%, 1.6% and 10.7% at 0.5B, 1.5B, 3B and 7B, with a worst single case of 37%. At 8192 tokens the 3B Q4_K_M model is 16% slower where the byte ratio predicts 14%. Attention arithmetic grows with depth as well, to 39% of the weight FLOPs for that model at 8192 tokens, and on a 16 GB machine the 7B runs at long context are close to the memory limit. Grouped-query attention keeps κ small in this family [31].

F. Batching hits a compute ceiling early

Batching is the standard way out of the bandwidth bound: B sequences share one read of the weights, so total throughput should rise with B until compute runs out [5]. Fig. 7 shows that on this stack compute runs out almost immediately. With 16-bit weights (3B only), throughput rises 3.6 $\times$ from $B = 1$ to $B = 4$ and reaches only 4.2 $\times$ at $B = 8$. With Q8_0 it is 2.2 to 2.5 $\times$ higher at $B = 8$ than at $B = 1$, and with Q4_K_M only 1.1 to 1.3 $\times$. For more than eight sequences the Metal backend switches from matrix-vector to matrix-matrix kernels (the threshold is a constant in `ggml-metal-common.cpp` at the commit we used) and throughput climbs again, to 5.1 to 7.0 $\times$ at $B = 32$ for Q4_K_M.

Our first reading of the plateau was that the quantized kernels re-read the weights for every sequence. The data rules that out. At the plateau, 3B Q8_0 produces 76 tokens/s; re-reading W for each would need 250 GB/s from a 107 GB/s interface. The reads are shared. What the plateau configurations have in common is their FLOP rate, 0.38 to 0.49 TFLOP/s, whatever the format or size. The small-batch kernels are compute-bound there. Formats that start closer to that ceiling at $B = 1$ have less room to gain, which is why Q4_K_M gains least.

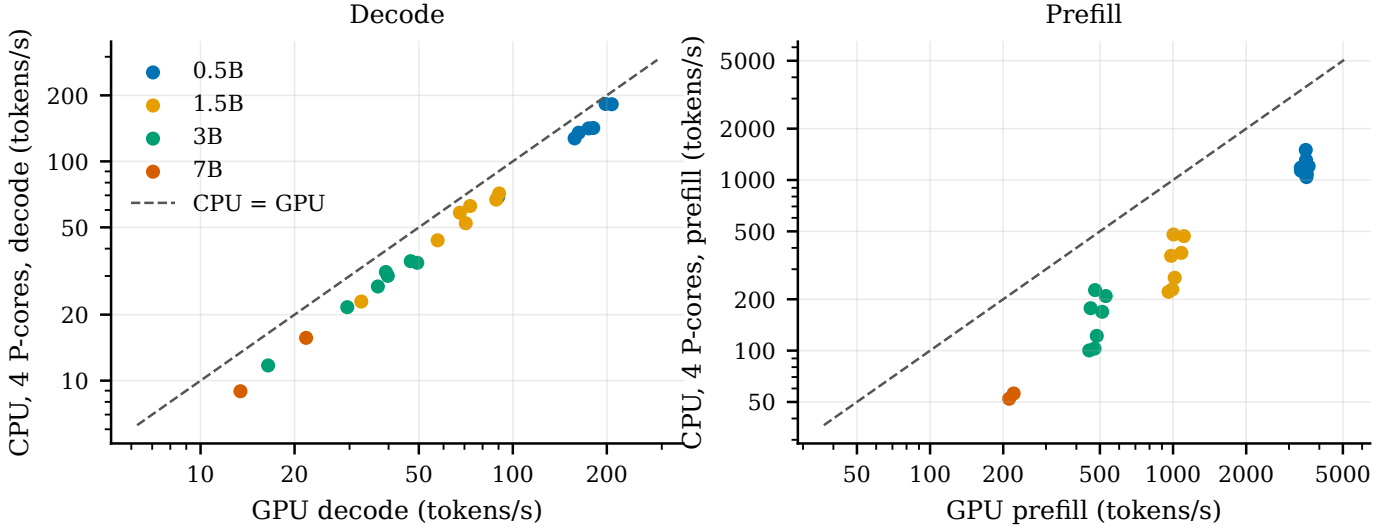


Fig. 4. The same configurations on the CPU (four performance cores, default build with Accelerate) and the GPU.

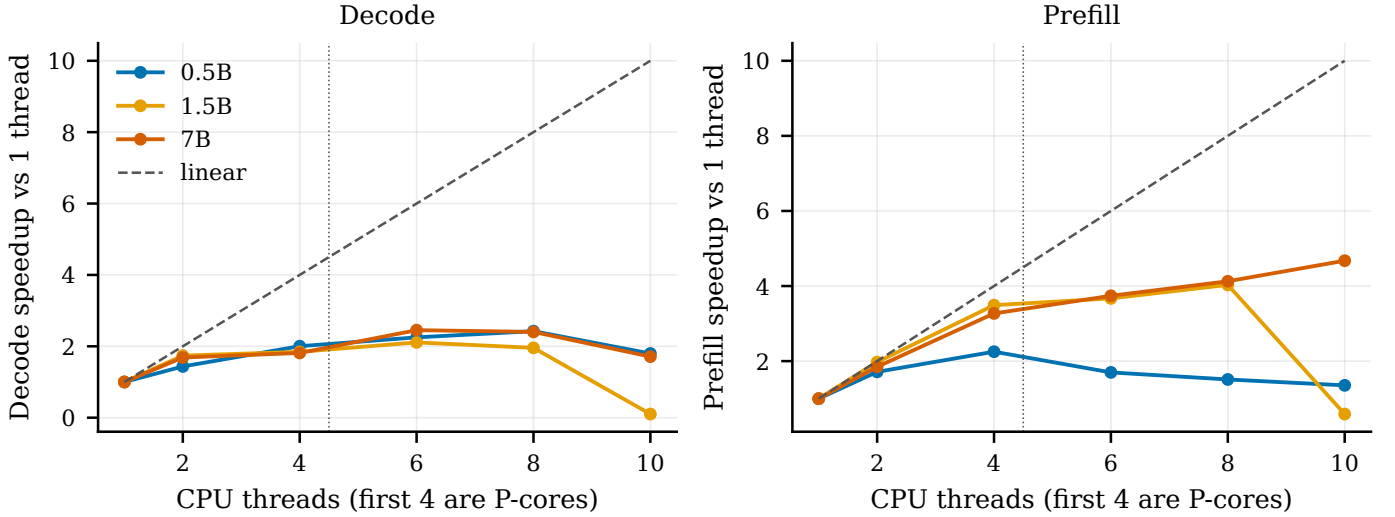


Fig. 5. CPU speedup over one thread, Q4_K_M. Threads beyond four run on efficiency cores.

For a single user this matters because speculative decoding is the one way to create a small batch. On this machine and software, a batch of 8 buys 1.1 to 1.3 $\times$ for Q4_K_M and 4.2 $\times$ for 16-bit weights, not 8 $\times$. The batch data covers 7 runs; four more were set aside because their $B = 1$ rate differed by more than 5% from the core measurement of the same configuration, which we take as a sign of background load.

G. Throttling as a probe

The test machine has no fan, and the first pass of our sweep ran hot. Low-bit configurations lost up to a third of their speed within a run while 16-bit decode did not move. We turned that observation into a controlled experiment, because it can separate two explanations that the byte counts cannot: a format that is slow because it costs arithmetic should slow further when the processor is throttled, and one that is slow because it uses the memory interface poorly should not.

The probe measures 3B decode in three formats on a machine that has idled for five minutes, runs a 300 s compute-bound load, and measures again. Every hot measurement is preceded by the same 25 s burst of load, the hot round is run twice in opposite format order, and each hot decode is followed by a short prefill as a control. Memory bandwidth is measured cold and hot with the same kernel as in Section III.

Hot against cold, memory bandwidth changes by -0.7% (copy) and -2.8% (triad). The prefill control changes by -22%, -45% and -40% for the three formats. The chip was throttled during five of the six hot measurements; the first 16-bit one was not (its control was down 2%), and the second was (down 42%, with 16-bit decode unchanged). Batch-1 decode changes by 0% for 16-bit, -2% for Q8_0 and -16% for Q4_K_M. Decode at $B = 4$, which Section IV-F argues is compute-bound, changes by -5%, -19% and -22%.

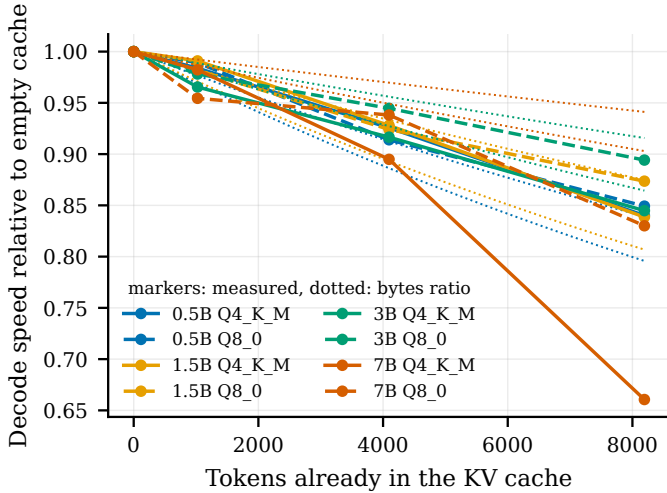


Fig. 6. Decode speed against context depth, relative to an empty cache. Dotted lines are the ratio of bytes read, $W/(W + \kappa L)$, with nothing fitted.

The premise holds: throttling changed the compute-bound control by -22% to -45% and memory bandwidth by less than 3%. The response of decode then follows the roofline of Section IV. Sixteen-bit and Q8_0 decode, at 1.0 and 1.9 FLOP per byte, sit well left of the decode ridge at 4.4, and they did not slow down. Q4_K_M, at 3.2 FLOP per byte, sits under the ridge but close to it. During the heating phase its batch-1 decode held its speed (-1% over the run) while prefill in the same loop lost 28%, and it gave way only when the control had fallen by about 40%, at the very end of the heating phase and in the hot rounds. If the decode ceiling fell in proportion to the control, the roofline predicts changes of 0%, 0% and -17% for the three formats; we measured 0%, -2% and -16%. At $B = 4$, where the quantized formats are on the compute plateau of Section IV-F, Q8_0 and Q4_K_M changed by -19% and -22%, and 16-bit, which has not reached the plateau at $B = 4$, by -5%.

This is two hot rounds on one model, the throttle state was not constant (the 16-bit control was nearly unthrottled in the first round and -22% on average), and the prefill control is only a proxy for how far the decode kernels slowed. With those limits, the probe supports two statements. Batch-1 decode of 16-bit and 8-bit weights on this machine is limited by memory and not by compute. And at least part of what Q4_K_M loses against its bandwidth line is compute: it disappears into the ceiling when the ceiling comes down.

H. An unplanned second probe

A second natural experiment arrived uninvited. After the laptop was moved to wall power, macOS started its idle-time maintenance jobs (Photos analysis and two text-indexing services), one of them at more than a full core. A repeat of the core sweep (18 of the 23 configurations) taken while they ran came out a median 10% slower in decode over the 11 configurations of the 1.5B and 3B models (range 3 to 14%), with prefill changed by -1%; the seven 0.5B configurations

TABLE I
MEAN ABSOLUTE PERCENTAGE ERROR OF PREDICTED DECODE THROUGHPUT (WORST CASE IN PARENTHESES). CONSTANTS ARE FITTED ON THE TRAINING SIZES ONLY.

Model	Train $n = 14$	1.5B (seen) $n = 7$	7B (blind) $n = 2$
M0: BW/W	36.0 (72)	37.2 (79)	9.5 (13)
M1: η_m	14.3 (24)	17.8 (29)	21.5 (24)
M2: η_m, τ	7.9 (27)	12.2 (27)	8.6 (9)
M3: M2 + k_q	3.7 (13)	4.3 (10)	3.3 (5)

in the same sweep, which are dominated by fixed overheads, were within about 2% of their final values. A back-to-back test on 3B 16-bit confirmed the cause: 13.8 tokens/s with the jobs running, 15.1 with the largest one paused and 16.5 with all three paused, while prefill moved by -3%. Throttling takes away compute and leaves 16-bit decode alone; background memory traffic takes away bandwidth and leaves prefill alone. We discarded the contended sweep, paused those jobs for the final measurements, and from then on recorded the busiest unrelated process with every run.

All other measurements in this paper were taken with 60 s of idle time before each configuration and repeated after a longer pause if the spread across repetitions exceeded 5%. The largest spread in the final GPU dataset is 2.3%, and in the four-thread CPU dataset 4.6%.

V. HOW GOOD IS THE FIRST-ORDER MODEL?

A. What was fitted on what

The constants are fitted on the 0.5B and 3B models (14 configurations). The other two sizes are not equally clean tests, and we report them separately. The 1.5B model (7 configurations) was never used for fitting, but its residuals were on screen while we decided which model variants to try; the per-format variant below was added because of them. The 7B model (2 configurations) was measured after the model forms were fixed. The only later change was an accounting correction to how prefill FLOPs are counted, made without reference to the 7B data. It is a blind test, a small one, and it is the largest model.

B. A ladder of models

Table I adds one ingredient at a time. **M0** is BW/W with no constants. **M1** fits only the bandwidth efficiency η_m . **M2** adds the per-layer overhead τ ; the compute rate C comes from prefill and is not fitted to decode. The additive form had the lower training error (7.9% against 10.4%) and is the one reported. **M3** adds one multiplier per weight format.

Three things stand out. First, M0 is poor on small models and better on the large one: 37.2% at 1.5B against 9.5% at 7B. The comparison flatters 7B, which has only Q8_0 and Q4_K_M; for those two formats alone M0 is off by 18.6% at 1.5B and 14.5% at 3B. Bandwidth over bytes is always optimistic, and it gets better as the weight read grows relative to everything else in a step. Second, a bandwidth efficiency alone (M1) is worse than nothing on the blind test. One

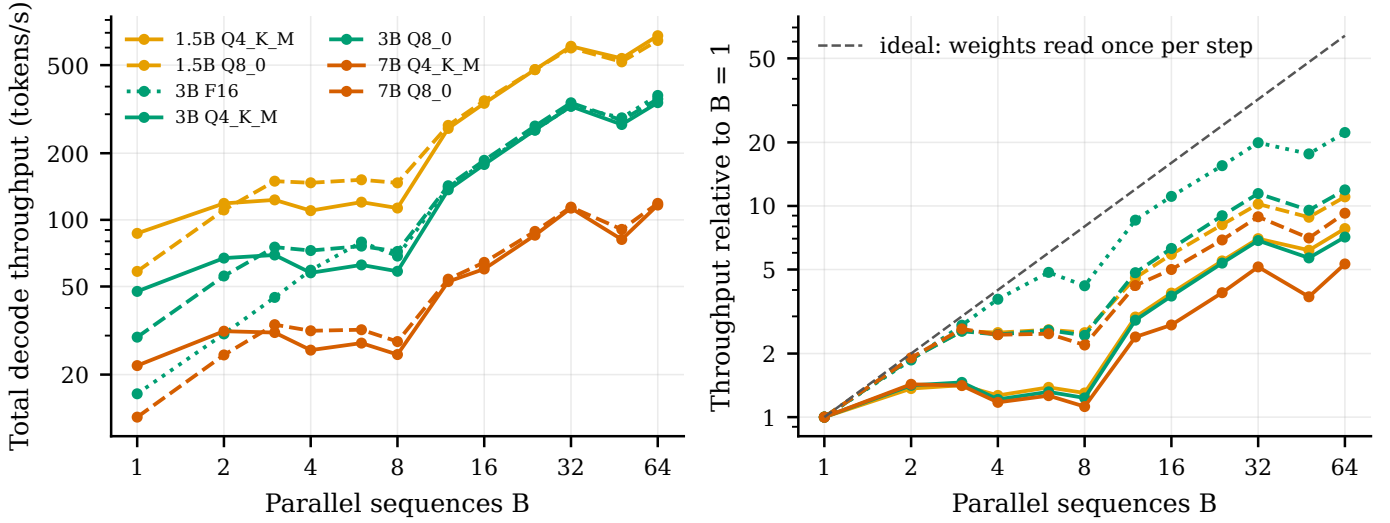


Fig. 7. Total decode throughput against the number of parallel sequences. Left: absolute. Right: relative to $B = 1$, with the ideal line for weights read once per step and unlimited compute.

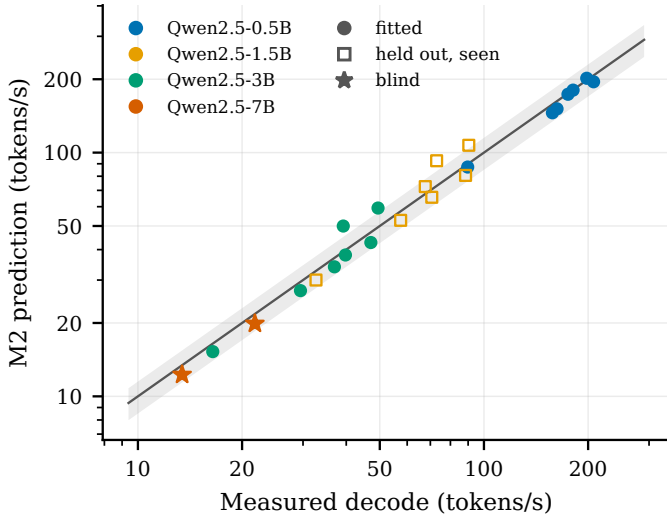


Fig. 8. Predicted against measured decode throughput for M2. Filled circles were used for fitting, open squares are the 1.5B model, and stars are the blind 7B test. The band is $\pm 15\%$.

constant cannot serve both a 0.5B model, where by M2's estimate fixed overheads are 11 to 26% of the step, and a 7B model, where they are 2 to 3%; the fit splits the difference and misses both. The per-layer overhead is what makes the model transfer across sizes: M2 has $\eta_m = 0.94$ and $\tau = 53 \mu\text{s}$ per layer and is off by 8.6% on 7B. Third, M2's remaining error is sorted by format, not by size (Fig. 8). At 1.5B and 3B it overpredicts Q2_K by 19 to 20% and Q3_K_M by 27%, and underpredicts 16-bit, Q8_0, Q6_K and Q4_K_M alike by 7 to 9%. Leaving one format out of the fit and predicting it at all sizes gives errors from 4% to 23%, worst for Q3_K_M.

C. One multiplier per format

M3 multiplies the compute term by a constant k_q for each format, with $k_{F16} = 1$, fitted on the training sizes and applied unchanged elsewhere. Error falls to 4.3% at 1.5B and 3.3% on the blind 7B configurations (-5.0% for Q4_K_M and -1.6% for Q8_0). The fitted values are $k_q = 0.7, 0.8, 1.4, 1.1, 3.0$ and 2.6 for Q8_0, Q6_K, Q5_K_M, Q4_K_M, Q3_K_M and Q2_K.

We do not read these as arithmetic costs. For a fixed format, the memory term and the compute term both scale with the parameter count, so across model sizes a larger k_q cannot be told apart from a lower bandwidth efficiency for that format. The data says that Q3_K_M and Q2_K carry two to three times the unexplained time per weight that Q4_K_M does. It does not say whether that time goes to unpacking arithmetic or to a worse access pattern. Two things show the constants are not physical on their own. The values for Q8_0 and Q6_K are below 1, which would make an 8-bit block cheaper to process than a plain 16-bit multiply-accumulate. And once the format constants are in the model, η_m moves from 0.94 to 0.99: what M2 charged to the interface, M3 charges to the formats. The throttling probe of Section IV-G is the experiment that can separate the two explanations, because it changes the compute rate and leaves the memory alone. All of these constants describe one llama.cpp commit on one GPU.

D. What the model gets wrong

It has a single compute rate, taken from prefill, while decode runs on kernels with a ceiling $6\times$ lower (Section IV). At batch 1 that barely matters for 16-bit weights and matters for 4-bit ones, which sit near the decode ridge. It treats formats as byte counts unless given a per-format constant whose meaning it cannot identify. And it knows nothing about temperature: on a fanless machine C changes within minutes.

TABLE II

DEVICE PARAMETERS USED. VALUES MARKED $\dagger$ ARE OUR ARITHMETIC ON A PUBLISHED FIGURE (THE PER-STACK HBM-PIM BANDWIDTHS DIVIDE A FOUR-STACK TOTAL; THE HBM-PIM PIM-CAPABLE CAPACITY FOLLOWS FROM A DIE DESCRIPTION IN [15]; THE UPMEM FIGURES MULTIPLY PER-CORE VALUES BY EIGHT). BALANCE IS COMPUTE THROUGHPUT DIVIDED BY INTERNAL BANDWIDTH.

Device	BW_{ext} (GB/s)	BW_{int} (GB/s)	r	Balance (FLOP/B)	Compute datatype
HBM-PIM stack [8], [18]	307 $\dagger$	1230 $\dagger$	4	0.98	FP16
GDDR6-AiM chip [19], [14]	64	1024	16	0.98	BF16
UPMEM chip [11]	n/a	5.6 $\dagger$	n/a	0.02 $\dagger$	INT32

VI. WHAT PROCESSING-IN-MEMORY WOULD CHANGE

Everything in this section is a projection from published device parameters (Table II). We have no PIM hardware.

A. What the parts have in common

HBM-PIM and AiM put multiply-accumulate units next to the DRAM banks, so a matrix-vector product reads its weights without crossing the pins. The bandwidth available to those units is $r = 4$ times the external interface for HBM-PIM, by Samsung’s statement [8], and $r = 16$ for AiM, from 1024 GB/s internal [14] against 64 GB/s at the pins [19]. Two properties matter for what follows. Both compute only in 16-bit floating point, FP16 for HBM-PIM and BF16 for AiM [18], [19]. And both are compute-balanced for exactly that case. A 16-bit matrix-vector product needs 1 FLOP per byte streamed (two operations per two-byte weight). HBM-PIM’s 1.2 TFLOPS over 1230 GB/s is 0.98 FLOP per byte, and AiM’s 1 TFLOPS over 1024 GB/s is 0.98. Both parts carry just enough arithmetic for 16-bit weights and no more.

UPMEM shows the datatype problem from the other side. Its cores have no full hardware multiplier [11]; from the published multiply and add rates, one multiply-accumulate runs at about 8.7 million per second per core. That is 0.02 FLOP per byte of internal bandwidth, so a 32-bit matrix-vector product over its memory would be compute-bound by a factor of about 20.

B. The datatype mismatch

Keep the memory technology fixed and ask what PIM adds. We split each measured host step into the weight read, $W/(\eta_m BW)$ with M2’s η_m , and everything else. PIM divides the weight read by r , subject to its compute balance, and leaves the rest alone. We give the PIM side the smaller “everything else” of the 16-bit host, which has no unpacking cost. This keeps the host’s fixed overheads in the comparison. Dividing r by the measured quantization speedup would scale those overheads by r as well, and gives inflated figures (1.4 to 2.0 $\times$ for $r = 4$).

Against a host running the same 16-bit weights, a 16-bit PIM with $r = 4$ is 2.9 to 3.5 $\times$ faster across the 3 model sizes where we have both formats. But the host is not running 16-bit weights. Q4_K_M is already 2.0 to 2.9 $\times$ faster than 16-bit on the same machine. Against that host, the same PIM is 1.2 to 1.4 $\times$ faster, and with $r = 16$ it is 2.8 to 3.6 $\times$ (Fig. 9). It also

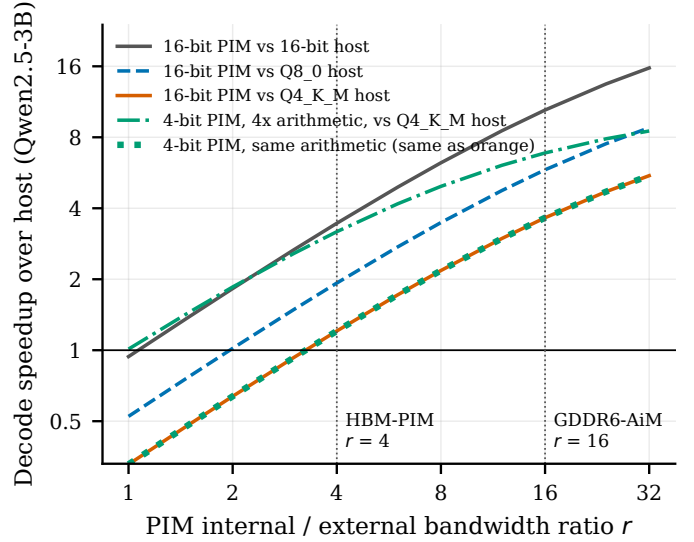


Fig. 9. Projected batch-1 decode speedup from adding PIM to the same memory technology, against the internal-to-external bandwidth ratio r , for the largest model measured in both formats. Only the weight read is scaled; the host’s measured overheads are kept. Vertical lines mark the two commercial DRAM-PIM parts.

needs 2.5 to 3.2 $\times$ the memory for the same model, on parts that give up 25 to 75% of their density to logic according to [15].

C. Would a 4-bit PIM fix it?

The obvious answer is a PIM that computes on 4-bit weights. The compute balance says it is not that simple. A 4-bit format packs 3.2 FLOP into each byte, not 1, so a unit provisioned at 0.98 FLOP per byte is compute-bound by about that factor. With the same multiply-accumulate throughput, a 4-bit PIM is no faster than the 16-bit one: 1.2 to 1.4 $\times$ over the Q4_K_M host at $r = 4$. It saves capacity and nothing else. With four times the throughput, the gain rises to 2.4 to 3.2 $\times$ at $r = 4$ and 3.5 to 6.9 $\times$ at $r = 16$. Low-bit integer arithmetic is cheap, 0.2 pJ for an 8-bit multiply against 1.1 pJ for a 16-bit floating-point one in Horowitz’s table [25], so more units per bank is a plausible direction, but the unpacking of block formats has to happen next to the bank as well, and Section V shows it is not free on a GPU. We do not know what it would cost in a DRAM process.

D. Batching

A host reads the weights once per step however many sequences it serves. A bank-level unit holds one input vector and streams the weights past it, so it runs one matrix-vector product per sequence [19], [13], and being compute-balanced it has no slack to do more. Against an ideal host in the same format, PIM’s advantage is gone at $B = r$. The datacenter proposals reflect this: NeuPIMs gives PIM the matrix-vector work and an NPU the batched matrix-matrix work [13]. Our measured host is far from ideal at small batch (Section IV-F), which helps PIM, and a single user has at most a small

speculative batch. Batch 1 is where PIM helps most and it is most of what a single user runs.

E. A cross-check

IANUS reports 29.9 ms per token on an A100 for a 2.5B-parameter GPT-2 in BF16 at batch 1 [14]. Bandwidth over bytes for that model on an 80 GB A100 [20] is 2.5 ms; we do not know which A100 variant they used. With the 1555 GB/s that NeuPIMs quotes for an A100 [13] it would be a third longer. Either way the GPU is 9 to 12 $\times$ slower than its bandwidth line. IANUS’s own 5.7 ms for this model is 2.3 $\times$ above the A100’s line as well, so its speedup in this case (5.2 $\times$; the 6.2 $\times$ is an average over model sizes) is not a bandwidth ratio. A datacenter GPU serving one short sequence of a small model is dominated by per-step overheads, the term our τ captures on the M4. First-order models like ours will understate PIM’s advantage over a baseline that is overhead-bound and overstate it over one that is tuned and quantized.

F. Energy

We did not measure power, so this is arithmetic on published constants, good to an order of magnitude. Moving the Q4_K_M weights of the 7B model once costs 137 mJ at the 3.92 pJ/bit modelled for an HBM2 access [27], and 699 mJ at the “over 20 pJ/bit” Horowitz gives for DRAM I/O [25]. We assume the M4’s memory lies somewhere in that range; we found no citable figure for it. The bare arithmetic for one token, at 1.5 pJ per 16-bit multiply plus add in Horowitz’s 45 nm table, is 11 mJ. A programmable processor pays far more than the bare operation: at his 70 pJ per instruction the same token costs 495 mJ. So data movement is one to two orders of magnitude above the arithmetic itself and comparable to what a real processor spends around it. O’Connor et al. attribute 65% of an HBM2 access to moving data from the sense amplifiers to the package [27]. Near-bank computation avoids most of that share, and Samsung reports 3.5 $\times$ lower energy per bit transferred for HBM-PIM [9].

VII. LIMITATIONS

One machine, one software stack, one model family.

Every measurement comes from a single MacBook Air, one llama.cpp commit and the Qwen2.5 architecture. The per-format constants, the decode compute ceiling and the batch threshold are properties of that software on that GPU. The bandwidth-bound behaviour of 16-bit decode should transfer; the rest should be re-measured.

A passively cooled laptop with a busy operating system.

Sustained load throttles this machine within minutes, and all-core CPU load within seconds. Background jobs that start on wall power cut decode at 1.5B and 3B by a median 10% in a full sweep and by 16% in a back-to-back test. We found both the hard way. The final GPU core dataset (all 23 configurations) was taken with a cooldown before each configuration and a 5% spread limit; its largest spread is 2.3%. Eighteen of the configurations were re-measured at the end with a desk fan pointed at the laptop, the interfering jobs

paused and background load logged. The other five (3B Q2_K, Q3_K_M and Q5_K_M on battery, and both 7B configurations on wall power) come from an earlier cooled pass without that logging, so the blind test rests on runs we could not check for background load directly. The CPU, thread, depth and batch runs were taken earlier, some on battery and some on wall power, before we logged background load. We checked them after the fact: every depth run’s empty-cache rate is within 5% of the core measurement of the same configuration (worst 4.0%), four batch runs failed that test and were excluded, and the CPU results agree within 3% between two independent stages at 1.5B and 7B (10% at 0.5B). The thread-scaling data remains noisy. We did not record whether the fan was running during the throttling probe, so the probe may understate how far this machine throttles on its own. Of the 79 result files in the final dataset, 15 were recorded on battery power and 64 on wall power, with low-power mode off; one configuration measured cooled on both sources differs by under 1%. The throttled first pass, the contended sweep and the excluded runs are all kept in the repository.

Bandwidth was measured from the CPU. Our *BW* is a CPU-side measurement whose two kernels differ by about 10%, and that uncertainty carries into every “percent of the line” figure. The GPU may see a different bandwidth; any difference is absorbed into η_m . We did not measure power.

A small blind test. The only configurations that no modelling decision saw are the two 7B ones. The 1.5B results are held out from fitting but were visible during model design.

The projection is first-order. It ignores DRAM command timing, row activation, the capacity PIM gives up to logic, the gathering of partial results between chips, and where attention and the KV cache run, which is the central question of AttAcc and NeuPIMs [12], [13]. It applies the internal-to-external ratio of HBM and GDDR6 parts to a laptop’s memory, which no vendor has built. Several device parameters come from vendor slides or from papers reporting another party’s hardware, and some are our arithmetic on those; Table II marks them and the released hardware.yaml gives the source and a supporting quotation for each. The real AiM prototype ran at one eighth of its rated pin speed [19].

Speed only. We did not measure perplexity, so this paper cannot rank Q4_K_M against Q3_K_M as a choice.

VIII. CONCLUSION

On this laptop, generating a token means reading the model once, and for 16-bit weights the time that takes is set by memory bandwidth to within 5 to 6% at 1.5B and 3B. Bandwidth over bytes is a good estimate for large models and a poor one for small, heavily quantized ones. A bandwidth efficiency and a per-layer overhead, fitted on two model sizes, predict a third and a blind fourth with mean errors of 12.2% and 8.6% (worst case 27%). What is left is sorted by weight format, and our data cannot say whether it is arithmetic or access pattern. The decode kernels run out of compute at about 4.4 FLOP per byte, and 4-bit weights are at 3.2 at batch 1,

so small batches help much less than the roofline for prefill would suggest.

The published PIM parts are built for 16-bit weights, in datatype and in how much arithmetic they carry per byte of bandwidth. On-device inference has moved to 4 bits. A 16-bit part with $r = 4$ would gain 1.2 to 1.4 $\times$ over the 4-bit host we measured and need about three times the memory, and a 4-bit part with the same arithmetic would gain no more. For a single user, PIM needs low-bit multiply-accumulate next to the bank, and three to four times as much of it per byte of bandwidth as today's parts carry.

DATA AND CODE AVAILABILITY

The repository for this paper contains the benchmark scripts, all raw result files including the throttled first pass, the sourced hardware parameter table, the model and projection code, and a Makefile that regenerates every figure and every number in the text from the raw data.

REFERENCES

- [1] J. Backus, "Can programming be liberated from the von Neumann style? A functional style and its algebra of programs," *Communications of the ACM*, vol. 21, no. 8, pp. 613–641, Aug. 1978.
- [2] J. von Neumann, "First draft of a report on the EDVAC," *IEEE Annals of the History of Computing*, vol. 15, no. 4, pp. 27–75, 1993, reprint of the 1945 Moore School report.
- [3] W. A. Wulf and S. A. McKee, "Hitting the memory wall: Implications of the obvious," *ACM SIGARCH Computer Architecture News*, vol. 23, no. 1, pp. 20–24, Mar. 1995.
- [4] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, "AI and memory wall," *IEEE Micro*, vol. 44, no. 3, pp. 33–39, May 2024.
- [5] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 5, 2023.
- [6] Z. Yuan, Y. Shang, Y. Zhou, Z. Dong, Z. Zhou, C. Xue, B. Wu, Z. Li, Q. Gu, Y. J. Lee, Y. Yan, B. Chen, G. Sun, and K. Keutzer, "LLM inference unveiled: Survey and roofline model insights," 2024.
- [7] O. Mutlu, S. Ghose, J. Gómez-Luna, R. Ausavarungnirun, M. Sadrosadati, and G. F. Oliveira, "A modern primer on processing in memory," 2020.
- [8] Y.-C. Kwon, S. H. Lee, J. Lee, S.-H. Kwon, J. M. Ryu, J.-P. Son, S. O. H.-S. Yu, H. Lee, S. Y. Kim, Y. Cho, J. G. Kim, J. Choi, H.-S. Shin, J. Kim, B. Phuah, H. Kim, M. J. Song, A. Choi, D. Kim, S. Kim, E.-B. Kim, D. Wang, S. Kang, Y. Ro, S. Seo, J. Song, J. Youn, K. Sohn, and N. S. Kim, "A 20nm 6GB function-in-memory DRAM, based on HBM2 with a 1.2TFLOPS programmable computing unit using bank-level parallelism, for machine learning applications," in *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, Feb. 2021, pp. 350–352, paper 25.4.
- [9] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, S. O. A. Iyer, D. Wang, K. Sohn, and N. S. Kim, "Hardware architecture and software stack for PIM based on commercial DRAM technology: Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, Jun. 2021, pp. 43–56.
- [10] S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, N. Kim, Y. Kwon, V. Kornijcuk, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, J. Lee, D. Ko, Y. Jun, K. Cho, I. Kim, C. Song, C. Jeong, D. Kwon, J. Jang, I. Park, J. Chun, and J. Cho, "A 1ynm 1.25V 8Gb, 16Gb/s/pin GDDR6-based accelerator-in-memory supporting 1TFLOPS MAC operation and various activation functions for deep-learning applications," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, Feb. 2022, pp. 1–3.
- [11] J. Gómez-Luna, I. El Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, "Benchmarking a new paradigm: Experimental analysis and characterization of a real processing-in-memory system," *IEEE Access*, vol. 10, pp. 52 565–52 608, 2022.
- [12] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, "AttAcc! unleashing the power of PIM for batched transformer-based generative model inference," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 2, Apr. 2024, pp. 103–119.
- [13] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "NeuPIMs: NPU-PIM heterogeneous acceleration for batched LLM inferencing," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 3, Apr. 2024, pp. 722–737.
- [14] M. Seo, X. T. Nguyen, S. J. Hwang, Y. Kwon, G. Kim, C. Park, I. Kim, J. Park, J. Kim, W. Shin, J. Won, H. Choi, K. Kim, D. Kwon, C. Jeong, S. Lee, Y. Choi, W. Byun, S. Baek, H.-J. Lee, and J. Kim, "IANUS: Integrated accelerator based on NPU-PIM unified memory system," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 3, Apr. 2024, pp. 545–560.
- [15] Y. Gu, A. Khadem, S. Umesh, N. Liang, X. Servot, O. Mutlu, R. Iyer, and R. Das, "PIM is all you need: A CXL-enabled GPU-free system for large language model inference," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 2, Mar. 2025, pp. 862–881.
- [16] ggml-org, "llama.cpp: LLM inference in C/C++," GitHub repository, MIT license. k-quant formats introduced in pull request #1684 ("k-quants"), <https://github.com/ggml-org/llama.cpp/pull/1684>. Accessed 2026-09-20. [Online]. Available: <https://github.com/ggml-org/llama.cpp>
- [17] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.
- [18] J. H. Kim, S.-h. Kang, S. Lee, H. Kim, W. Song, Y. Ro, S. Lee, D. Wang, H. Shin, B. Phuah, J. Choi, J. So, Y. Cho, J. Song, J. Choi, J. Cho, K. Sohn, Y. Sohn, K. Park, and N. S. Kim, "Aquabolt-XL: Samsung HBM2-PIM with in-memory processing for ML accelerators and beyond," in *2021 IEEE Hot Chips 33 Symposium (HCS)*, Aug. 2021, pp. 1–26.
- [19] Y. Kwon, V. Kornijcuk, N. Kim, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, G. Kim, B. An, J. Kim, J. Lee, I. Kim, J. Park, C. Park, Y. Song, B. Yang, H. Lee, S. Kim, D. Kwon, S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, M. Lee, M. Shin, M. Shin, J. Cha, C. Jung, K. Chang, C. Jeong, E. Lim, I. Park, and J. Chun, "System architecture and software stack for GDDR6-AiM," Poster, IEEE Hot Chips 34 Symposium (HCS), Aug. 2022, accessed 2026-09-20. [Online]. Available: <https://hc34.hotchips.org/assets/program/posters/hc34.SKhyunx.YongkeeKwon.v03.pdf>
- [20] NVIDIA Corporation, "NVIDIA A100 tensor core GPU datasheet," Datasheet, document nvidia-a100-datasheet-nvidia-us-2188504-web, a100 80GB PCIe and SXM specifications. Publication year not printed in the extracted text. Accessed 2026-09-20. [Online]. Available: <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-nvidia-us-2188504-web.pdf>
- [21] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for Transformer-based generative models," in *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Jul. 2022, pp. 521–538. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/you>
- [22] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, Oct. 2023, pp. 611–626.
- [23] E. Frantar, S. Ashkboos, T. Hoeffler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *International Conference on Learning Representations (ICLR)*, 2023.
- [24] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware weight quantization for LLM compression and acceleration," in *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

- [25] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, Feb. 2014, pp. 10–14, session 1.1.
- [26] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both weights and connections for efficient neural networks," in *Advances in Neural Information Processing Systems (NIPS)*, 2015, pages/volume not verified; cite via arXiv:1506.02626.
- [27] M. O'Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, "Fine-grained DRAM: Energy-efficient DRAM for extreme bandwidth systems," in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Oct. 2017, pp. 41–54.
- [28] C. Ortega, Y. Falevoz, and R. Aygnac, "PIM-AI: A novel architecture for high-efficiency LLM inference," 2024.
- [29] G. F. Oliveira, J. Gómez-Luna, L. Orosa, S. Ghose, N. Vijaykumar, I. Fernandez, M. Sadrosadati, and O. Mutlu, "DAMOV: A new methodology and benchmark suite for evaluating data movement bottlenecks," *IEEE Access*, vol. 9, pp. 134 457–134 502, 2021.
- [30] Apple Inc., "MacBook Air (15-inch, M4, 2025) – tech specs," Apple Support, 2025, accessed 2026-09-20. [Online]. Available: <https://support.apple.com/en-us/122210>
- [31] Qwen Team, "Qwen2.5 technical report," 2024, arXiv lists individual authors beginning An Yang, Baosong Yang, Beichen Zhang, et al.